

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Database Management Systems
COURSE CODE: CSA05

COURSE OUTCOME COVERED

CO5: *Implement database operations using query processing, optimization, and transaction management to ensure consistency and reliability.* (BL3, BL4)

Activity Tool : Constraint-Based Problem Solving (High)
Assessment Tool Weightage: 35%

Dr

QUESTIONS			Total Marks: 50
Q.No	Question	Bloom's Level	Marks
1	Given a complex SQL query with nested joins and sub-queries, draw the query execution plan and apply heuristic optimization rules step-by-step.	BL4 – Analyze	5
2	Apply cost-based optimization to choose between nested-loop join, sort-merge join, and hash join for the given relations with specified tuple counts and page sizes.	BL4 – Analyze	5
3	Implement JDBC connectivity in Java to a MySQL database and write code to execute parameterized queries with PreparedStatement.	BL3 – Apply	5
4	Given a schedule of four concurrent transactions, determine whether it is conflict-serializable using a precedence graph.	BL4 – Analyze	5
5	Demonstrate the application of Basic Two-Phase Locking (2PL) on a given transaction set. Identify and resolve any deadlocks.	BL3 – Apply	5
6	Apply the Wait-Die and Wound-Wait timestamp-based deadlock prevention schemes on a given set of transactions requesting locks.	BL4 – Analyze	5

7	Given transaction logs with checkpoints, apply the Immediate Update recovery protocol to recover the database after a system failure.	BL3 – Apply	5
8	Apply Deferred Update (NO-UNDO/REDO) recovery technique on a given log file with committed and uncommitted transactions.	BL3 – Apply	5
9	Design a concurrency control mechanism using strict 2PL for a banking system where T1 transfers funds and T2 reads balance simultaneously.	BL4 – Analyze	5
10	Write pseudocode for query optimization using heuristics: push down selection, project early, and order joins by smaller intermediate results.	BL4 – Analyze	5

Code of Conduct:

I K.MOHAMMED NAFEEZ [192571070] certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: K.MOHAMMED NAFEEZ

RUBRICS FOR CALCULATION:

Constraint-Based Problem Solving Assessment Rubric					
Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization

Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided

ANSWERS – CO5 CONSTRAINT-BASED PROBLEM SOLVING

Q1. Given a complex SQL query with nested joins and sub-queries, draw the query execution plan and apply heuristic optimization rules step-by-step.

Answer:

Consider the following example query:

```
SELECT s.name
FROM Student s
JOIN Enrollment e ON s.sid = e.sid
JOIN Course c ON e.cid = c.cid
WHERE s.dept = 'CSE'
      AND c.credits = 4
      AND s.sid IN (SELECT sid FROM Enrollment WHERE marks > 80);
```

Query execution and heuristic optimization:

1. Start with the relational algebra representation.
2. Convert the nested sub-query into a semi-join/selection condition where possible.
3. Push selections down to the base relations:
 $\sigma(\text{dept}='CSE')(\text{Student})$
 $\sigma(\text{credits}=4)(\text{Course})$
 $\sigma(\text{marks}>80)(\text{Enrollment})$
4. Push projections down to keep only required attributes:
 $\text{Student} \rightarrow \{\text{sid}, \text{name}\}$
 $\text{Enrollment} \rightarrow \{\text{sid}, \text{cid}, \text{marks}\}$
 $\text{Course} \rightarrow \{\text{cid}, \text{credits}\}$
5. Perform the most selective selections before joins.
6. Join the reduced Student and Enrollment relations.
7. Join the result with the reduced Course relation.
8. Apply the final projection to obtain student names.

Optimized plan:

```
 $\pi(\text{name})$ 
|
Join('Student', 'Enrollment')
|
Join with Course'
|
```

$\sigma(\text{credits}=4)(\text{Course})$
and
 $\sigma(\text{dept}=\text{'CSE'})(\text{Student})$
and
 $\sigma(\text{marks}>80)(\text{Enrollment})$

This reduces intermediate relations and therefore reduces disk I/O and processing cost.

Q2. Apply cost-based optimization to choose between nested-loop join, sort-merge join, and hash join for the given relations with specified tuple counts and page sizes.

Answer:

Assume relation R has 1,000 tuples occupying 100 pages and relation S has 10,000 tuples occupying 1,000 pages.

1. Nested-loop join:

A simple page nested-loop cost is approximately:

$$\begin{aligned}\text{Cost} &= B(R) + B(R) \times B(S) \\ &= 100 + (100 \times 1000) \\ &= 100,100 \text{ page I/Os.}\end{aligned}$$

This is expensive for large relations.

2. Sort-merge join:

The relations must be sorted and then merged. It is useful when inputs are already sorted or when the join result is large. Its cost is approximately the cost of external sorting both relations plus the merge cost.

3. Hash join:

For an equality join, hash join partitions both relations and matches tuples having the same hash value. It normally requires roughly linear I/O in the relation sizes when sufficient memory is available.

Decision:

For large, unsorted relations and an equality condition, choose HASH JOIN because it generally has the lowest practical cost. If the relations are already sorted or the join condition is suitable for ordered processing, SORT-MERGE JOIN can be preferred. NESTED-LOOP JOIN is best mainly when one relation is very small or an efficient index exists on the inner relation.

Therefore, under the assumed large-relation equality-join case, the cost-based optimizer should select HASH JOIN.

Q3. Implement JDBC connectivity in Java to a MySQL database and write code to execute parameterized queries with PreparedStatement.

Answer:

JDBC allows a Java application to communicate with MySQL.

Requirements:

- MySQL server
- MySQL JDBC driver
- Database and table
- Java program

Example table:

```
CREATE DATABASE college;
```

```
USE college;
```

```
CREATE TABLE student (  
    sid INT PRIMARY KEY,  
    name VARCHAR(50),  
    dept VARCHAR(30)  
);
```

Java code:

```
import java.sql.*;
```

```
public class JDBCExample {  
    public static void main(String[] args) {  
        String url = "jdbc:mysql://localhost:3306/college";  
        String user = "root";  
        String password = "your_password";  
  
        String sql = "SELECT sid, name, dept FROM student WHERE dept = ?";  
  
        try (Connection con = DriverManager.getConnection(url, user, password);  
            PreparedStatement ps = con.prepareStatement(sql)) {  
  
            ps.setString(1, "CSE");
```

```

ResultSet rs = ps.executeQuery();

while (rs.next()) {
    System.out.println(
        rs.getInt("sid") + " " +
        rs.getString("name") + " " +
        rs.getString("dept")
    );
}
} catch (SQLException e) {
    e.printStackTrace();
}
}
}

```

PreparedStatement uses the ? placeholder and binds the value with setString(). This is safer and more efficient for repeated parameterized queries than constructing SQL by string concatenation.

Q4. Given a schedule of four concurrent transactions, determine whether it is conflict-serializable using a precedence graph.

Answer:

A precedence graph contains:

- One node for every transaction.
- A directed edge $T_i \rightarrow T_j$ if T_i performs an operation before T_j on the same data item and the two operations conflict.

Example schedule:

T1: R1(A), W1(A)

T2: R2(A), W2(B)

T3: R3(B), W3(B)

T4: R4(C), W4(C)

Conflicting operations create edges:

$T1 \rightarrow T2$ because W1(A) occurs before R2(A).

$T2 \rightarrow T3$ because W2(B) occurs before R3(B).

No conflict exists between T4 and the other transactions because it uses C.

Precedence graph:

$T1 \rightarrow T2 \rightarrow T3$

T4

The graph contains no cycle. Therefore, the schedule is conflict-serializable.

An equivalent serial order is:

$T1 \rightarrow T2 \rightarrow T3 \rightarrow T4$

If a cycle such as $T1 \rightarrow T2 \rightarrow T3 \rightarrow T1$ existed, the schedule would not be conflict-serializable.

Q5. Demonstrate the application of Basic Two-Phase Locking (2PL) on a given transaction set. Identify and resolve any deadlocks.

Answer:

Basic 2PL has two phases:

1. Growing phase – a transaction may acquire locks but cannot release locks.
2. Shrinking phase – after releasing the first lock, the transaction cannot acquire any new lock.

Example:

T1 needs A and B.

T2 needs B and A.

Execution:

T1: LOCK-X(A)

T2: LOCK-X(B)

T1: requests LOCK-X(B) → waits because T2 holds B.

T2: requests LOCK-X(A) → waits because T1 holds A.

This creates a deadlock:

T1 waits for T2

T2 waits for T1

Deadlock resolution:

- Detect the cycle in the wait-for graph.
- Abort one transaction, for example T2.
- Release T2's locks.
- Allow T1 to obtain B and complete.
- Restart T2 after T1 commits.

Thus, 2PL guarantees conflict serializability, but basic 2PL can produce deadlocks. Deadlock detection and transaction rollback can resolve them.

Q6. Apply the Wait-Die and Wound-Wait timestamp-based deadlock prevention schemes on a given set of transactions requesting locks.

Answer:

Assume:

T1 has timestamp 10 (older)

T2 has timestamp 20 (younger)

T1 holds lock A and T2 requests A.

Wait-Die:

- Older transaction may wait for a younger transaction.
- Younger transaction requesting a lock held by an older transaction is aborted.

Therefore:

T2 requests A held by T1 → T2 is younger → T2 is rolled back.

If T1 requested a lock held by T2:

T1 is older → T1 may wait.

Wound-Wait:

- Older transaction can force a younger transaction to roll back.
- Younger transaction waits for an older transaction.

Therefore:

T1 requests A held by T2 → T1 is older → T2 is rolled back.

T2 requests A held by T1 → T2 is younger → T2 waits.

Comparison:

Wait-Die: older waits, younger dies.

Wound-Wait: older wounds/aborts younger, younger waits.

Both schemes prevent deadlock by using transaction timestamps to control waiting and rollback.

Q7. Given transaction logs with checkpoints, apply the Immediate Update recovery protocol to recover the database after a system failure.

Answer:

Immediate Update allows database changes to be written to disk before the transaction commits. Therefore recovery may require both UNDO and REDO.

Example log:

<START T1>
<T1, A, 100, 150>
<START T2>
<T2, B, 200, 250>
<CHECKPOINT>
<T1 COMMIT>
<T2, C, 300, 350>
<System Failure>

Recovery:

1. Identify committed transactions: T1.
2. Identify active/uncommitted transactions at failure: T2.
3. REDO committed transactions when required by the recovery algorithm:
REDO T1 changes.
4. UNDO uncommitted transaction T2:
restore B from 250 to 200 and C from 350 to 300, according to the log.
5. Complete recovery and write the necessary termination/abort records.

The checkpoint reduces the amount of log that must be examined. Immediate Update therefore uses both REDO for committed work and UNDO for uncommitted work.

Q8. Apply Deferred Update (NO-UNDO/REDO) recovery technique on a given log file with committed and uncommitted transactions.

Answer:

In deferred update, database changes are not written to the database until the transaction commits. Therefore uncommitted changes do not need to be undone.

Example log:

<START T1>
<T1, A, 100, 150>
<START T2>
<T2, B, 200, 250>
<T1 COMMIT>
<System Failure>

Recovery:

1. Identify committed transaction T1.
2. Identify uncommitted transaction T2.
3. REDO T1 because its changes must be reflected in the database.
4. Ignore T2 because its changes were never written to the database.
5. No UNDO operation is necessary.

Thus:

Committed transactions → REDO.

Uncommitted transactions → ignored.

UNDO → not required.

This is why the deferred-update method is called NO-UNDO/REDO.

Q9. Design a concurrency control mechanism using strict 2PL for a banking system where T1 transfers funds and T2 reads balance simultaneously.

Answer:

Strict 2PL keeps all exclusive locks until the transaction commits or aborts. This prevents other transactions from reading or modifying uncommitted data.

Suppose T1 transfers ₹1,000 from Account A to Account B.

T1:

1. LOCK-X(A)
2. LOCK-X(B)
3. READ(A)
4. $A = A - 1000$
5. WRITE(A)
6. READ(B)
7. $B = B + 1000$
8. WRITE(B)
9. COMMIT
10. UNLOCK(A), UNLOCK(B)

T2 wants to read A and B:

1. T2 requests LOCK-S(A) → waits because T1 has an exclusive lock.
2. T2 requests LOCK-S(B) → waits.
3. T1 commits.
4. T1 releases the locks.
5. T2 obtains shared locks and reads the new consistent balances.

This prevents dirty reads and ensures that T2 does not see only one side of the transfer. Strict 2PL therefore provides safe concurrency for banking transactions.

Q10. Write pseudocode for query optimization using heuristics: push down selection, project early, and order joins by smaller intermediate results.

Answer:

Pseudocode:

ALGORITHM HeuristicQueryOptimization(Query Q)

1. Convert SQL query Q into an initial relational algebra tree.
2. Push selection operations downward:
For every selection $\sigma(\text{condition})$:
Move σ as close as possible to the base relation.
3. Push projection operations downward:
For every projection $\pi(\text{attributes})$:
Keep only attributes required by selections, joins, and the final result.
4. Estimate the size of intermediate relations.
5. Arrange joins so that smaller intermediate results are produced as early as possible.
6. Prefer selective relations as early join inputs.
7. Rebuild the optimized relational algebra tree.
8. Return the optimized query execution plan.

Example:

Original:

$\pi(\text{attributes})(\sigma(\text{condition})(R \bowtie S \bowtie T))$

Optimized:

1. $\sigma(\text{condition})(R)$
2. $\pi(\text{required attributes})(R)$
3. Join reduced R with the smaller of S and T.
4. Join the result with the remaining relation.
5. Apply final projection.

Benefits:

- Fewer tuples are processed.

- Fewer attributes are carried through joins.
- Intermediate results become smaller.
- Disk I/O and CPU cost are reduced.
- Overall query execution becomes faster.